

```
# Encapsulation is process of binding data and method together in same class and t  
  
# Encapsulation is the process of hiding data using private variables and providi
```

```
class student:  
  
    def __init__(self,name,marks):  
        self.name = name  
        self.__marks = marks  
  
    def display(self):  
        return self.name  
  
    def get_marks(self):  
        return self.__marks
```

```
s1 = student("Rahul",90)  
print(s1.display())  
print(s1.get_marks())
```

```
Rahul  
90
```

```
class bank_account:  
    def __init__(self,balance):  
        self.balance = balance  
  
    def deposit(self,amount):  
        if amount > 0:  
            self.balance += amount  
            print(f"Rs {amount} was credit")  
            print(f"total balance is {self.balance}")  
        else:  
            print("invalid deposit amount")  
  
    def withdrawal(self,amount):  
        if amount <= self.balance:  
            self.balance -= amount  
        else:  
            print("insufficient balance")  
            print(f"Rs {amount} was debit ")  
            print(f"total balance is {self.balance}")  
  
    def get_balance(self):  
        return self.balance
```

```
bank1 = bank_account(2000)
print(bank1.balance)

bank1.deposit(4000)
print(bank1.balance)

bank1.withdrawal(4000)
print(bank1.balance)

print(bank1.get_balance())
```

```
2000
Rs 4000 was credit
total balance is 6000
6000
Rs 4000 was debit
total balance is 2000
2000
2000
```

```
class bank_account:
    def __init__(self,balance):
        self.__balance = balance

    def deposit(self,amount):
        if amount > 0:
            self.balance += amount
        else:
            print("invalid deposit amount")

    def withdrawal(self,amount):
        if amount <= self.balance:
            self.balance -= amount
        else:
            print("insufficient balance")

bank1 = bank_account(2000)
print(bank1.balance)

bank1.deposit(4000)
print(bank1.balance)

bank1.withdrawal(4000)
print(bank1.balance)
```

```

-----
AttributeError                                Traceback (most recent call last)
/tmp/ipython-input-2118231343.py in <cell line: 0>()
    18
    19 bank1 = bank_account(2000)
--> 20 print(bank1.balance)
    21
    22 bank1.deposit(4000)

AttributeError: 'bank_account' object has no attribute 'balance'

```

Next steps: [Explain error](#)

```

class bank_account:
    def __init__(self,balance):
        self.__balance = balance

    def deposit(self,amount):
        if amount > 0:
            self.__balance += amount
            print(f"Rs {amount} was debit")
        else:
            print("invalid deposit amount")

    def withdrawal(self,amount):
        if amount <= self.__balance:
            self.__balance -= amount
        else:
            print("insufficient balance")

    def get_balance(self):
        return self.__balance

bank1 = bank_account(2000)
print(f" initial balance is {bank1.get_balance()}")

bank1.deposit(4000)
print(f"check balance after deposit is {bank1.get_balance()}")

bank1.withdrawal(4000)
print(f"check balance after withdreawl is {bank1.get_balance()}")

print(f"total balance is {bank1.get_balance()}")

```

```

initial balance is 2000
Rs 4000 was debit
check balance after deposit is 6000
check balance after withdreawl is 2000
total balance is 2000

```

```
# employee salary system

class employee_salary_system:

    def __init__(self,name,salary):
        self.name = name
        self.salary = salary
        print("salary system database")

name = input("Enter any name is ")
salary = float(input("Enter a salary is "))
s1 = employee_salary_system(name,salary)
print(s1.name)
print(s1.salary)
```

```
Enter any name is Sinha rahul
Enter a salary is 7688
salary system database
Sinha rahul
7688.0
```

```
# employee salary system with encapsulation
```

```
class employee_salary_system:

    def __init__(self,name,salary):
        self.name = name
        self.__salary = salary
        print("salary system database")

name = input("Enter any name is ")
salary = float(input("Enter a salary is "))
s1 = employee_salary_system(name,salary)
print(s1.name)
print(s1.salary)
```

```
Enter any name is Rahul
Enter a salary is 8900
salary system database
Rahul
```

```
-----
AttributeError                                Traceback (most recent call last)
/tmp/ipython-input-2154944727.py in <cell line: 0>()
     12 s1 = employee_salary_system(name,salary)
     13 print(s1.name)
--> 14 print(s1.salary)
```

```
AttributeError: 'employee_salary_system' object has no attribute 'salary'
```

Next steps: [Explain error](#)

```
# employee salary system with encapsulation
```

```
class employee_salary_system:
```

```
    def __init__(self,name,salary):
        self.name = name
        self.__salary = salary
        print("salary system database")
```

```
    def show_salary(self):
        return self.__salary
```

```
name = input("Enter any name is ")
salary = float(input("Enter a salary is "))
s1 = employee_salary_system(name,salary)
print(s1.name)
print(s1.show_salary())
```

```
Enter any name is Ankit
Enter a salary is 7655
salary system database
Ankit
7655.0
```

```
# employee salary system with encapsulation
```

```
class employee_salary_system:
```

```
    def __init__(self, name, salary):
        self.name = name
        self.__salary = salary
        print("Salary system database created")

    def increment_salary(self, amount):
        if amount > 0:
            self.__salary += amount
            print(f"Rs {amount} is increment")
            print(f"total salary is {self.__salary}")
        else:
            print("Invalid increment amount")
```

```
    def show_salary(self):
        return self.__salary
```

```
name = input("Enter employee name: ")
salary = float(input("Enter employee salary: "))
amount = int(input("Enter increment amount: "))

s1 = employee_salary_system(name, salary)
```

```
print(f"Employee Name: {s1.name}")
print(f"Current Salary: Rs {s1.show_salary()}")

s1.increment_salary(amount)
print(f"Updated Salary: Rs {s1.show_salary()}")
```

```
Enter employee name: Ankush
Enter employee salary: 8000
Enter increment amount: 650
Salary system database created
Employee Name: Ankush
Current Salary: Rs 8000.0
Rs 650 is increment
total salary is 8650.0
Updated Salary: Rs 8650.0
```

```
class salary_system:

    def __init__(self,name,salary):
        self.name = name
        self.salary = salary
        print("employee_salary_system")

    def display_employee_name(self):
        return self.name,self.salary

name = input("Enter a name is ")
salary = float(input("Enter salary is "))
salary1 = salary_system(name,salary)
print(salary1.display_employee_name())
```

```
Enter a name is Ankita
Enter salary is 6500
employee_salary_system
('Ankita', 6500.0)
```

```
class salary_system:

    def __init__(self,name,salary):
        self.__name = name
        self.salary = salary
        print("employee_salary_system")

    def display_employee_name(self):
        return self.name,self.salary

name = input("Enter a name is ")
salary = float(input("Enter salary is "))
salary1 = salary_system(name,salary)
print(salary1.display_employee_name())
```

```
Enter a name is niraj
Enter salary is 9000
employee_salary_system
```

```
-----
AttributeError                                Traceback (most recent call last)
/tmp/ipython-input-817305191.py in <cell line: 0>()
    12 salary = float(input("Enter salary is "))
    13 salary1 = salary_system(name,salary)
--> 14 print(salary1.display_employee_name())

/tmp/ipython-input-817305191.py in display_employee_name(self)
     7
     8     def display_employee_name(self):
----> 9         return self.name,self.salary
    10
    11 name = input("Enter a name is ")

AttributeError: 'salary_system' object has no attribute 'name'
```

Next steps: [Explain error](#)

```
class salary_system:

    def __init__(self,name,salary):
        self.__name = name
        self.salary = salary
        print("employee_salary_system")

    def display_employee_name(self):
        return self.__name,self.salary

name = input("Enter a name is ")
salary = float(input("Enter salary is "))
salary1 = salary_system(name,salary)
print(salary1.display_employee_name())
```

```
Enter a name is Anuj
Enter salary is 9870
employee_salary_system
('Anuj', 9870.0)
```

```
# employee salary system with encapsulation

class salary_system:

    def __init__(self,name,salary):
        self.__name = name
        self.__salary = salary
        print("employee_salary_system")

    def display_employee_name(self):
```

```

        return self.__name,self.__salary

    def increment_salary1(self,amount):
        self.__salary += amount
        return self.__salary

    def get_salary(self):
        return self.__salary

name = input("Enter a name is ")
salary = float(input("Enter salary is "))
amount = float(input("Enter amount is "))
salary1 = salary_system(name,salary)
print(salary1.display_employee_name())
salary1.increment_salary1(amount)
print(salary1.display_employee_name())
print(f"total salary of {name} is {salary1.get_salary()}")

```

```

Enter a name is katan
Enter salary is 8900
Enter amount is 7655
employee_salary_system
('katan', 8900.0)
('katan', 16555.0)
total salary of katan is 16555.0

```

```

class bank_employee:

    def __init__(self,full_name,salary):
        self.full_name = full_name
        self.salary = salary
        print("bank_employee_databases")

    def display_employee_details(self):
        return self.full_name,self.salary

n = int(input("Enter a number: "))
employee = []

for i in range(n):

    full_name = input("Enter a name : ")
    salary = int(input("Enter a Employee_salary: "))
    Employee1 = bank_employee(full_name,salary)
    employee.append( Employee1)
    print(Employee1.display_employee_details())
    print(" ")

```

```

Enter a number: 1
Enter a name : Raj
Enter a Employee_salary: 7690

```



```
bank_employee_databases
('Raj', 7690)
```

```
class bank_employee:

    def __init__(self,full_name,salary):
        self.full_name = full_name
        self.__salary = salary
        print("bank_employee_databases")

    def display_employee_details(self):
        return self.full_name,self.salary

n = int(input("Enter a number: "))
employee = []

for i in range(n):

    full_name = input("Enter a name : ")
    salary = int(input("Enter a Employee_salary: "))
    Employee1 = bank_employee(full_name,salary)
    employee.append( Employee1)
    print(Employee1.display_employee_details())
    print(" ")
```

```
Enter a number: 1
Enter a name : Ramu
Enter a Employee_salary: 7800
bank_employee_databases
```

```
-----
AttributeError                                Traceback (most recent call last)
/tmp/ipython-input-3682064916.py in <cell line: 0>()
    18 Employee1 = bank_employee(full_name,salary)
    19 employee.append( Employee1)
--> 20 print(Employee1.display_employee_details())
    21 print(" ")

/tmp/ipython-input-3682064916.py in display_employee_details(self)
     7
     8 def display_employee_details(self):
--> 9     return self.full_name,self.salary
    10
    11 n = int(input("Enter a number: "))
```

```
AttributeError: 'bank_employee' object has no attribute 'salary'
```

Next steps: [Explain error](#)

```
class bank_employee:

    def __init__(self,full_name,salary):
        self.full_name = full_name
```

```

        self.full_name = full_name
        self.__salary = salary
        print("bank_employee_databases")

    def display_employee_details(self):
        return self.full_name,self.__salary

n = int(input("Enter a number: "))
employee = []

for i in range(n):
    print(f"number of employee is {i+1}")

    full_name = input("Enter a name : ")
    salary = int(input("Enter a Employee_salary: "))
    Employee1 = bank_employee(full_name,salary)
    employee.append( Employee1)
    print(Employee1.display_employee_details())
    print(" ")

```

```

Enter a number: 2
number of employee is 1
Enter a name : niketa
Enter a Employee_salary: 8900
bank_employee_databases
('niketa', 8900)

number of employee is 2
Enter a name : Jaya
Enter a Employee_salary: 7655
bank_employee_databases
('Jaya', 7655)

```

```

class bank_employee:

    def __init__(self,full_name,salary):
        self.full_name = full_name
        self.__salary = salary
        print("bank_employee_databases")

    def display_employee_details(self):
        return self.full_name,self.__salary

    def salary_increment(self,amount):
        self.__salary += amount
        print(f"RS {amount} is credit")
        print(f"total salary is {self.__salary}")

    def get_salary(self):
        return self.__salary

n = int(input("Enter a number: "))

```

```
employee = []

for i in range(n):
    print(f"\nEmployee {i+1}")
    full_name = input("Enter a name : ")
    salary = int(input("Enter a Employee_salary: "))
    amount = int(input("Enter a amount "))
    Employee1 = bank_employee(full_name,salary)
    employee.append( Employee1)
    print(Employee1.display_employee_details())
    Employee1.salary_increment(amount)
    print(Employee1.display_employee_details())
    print(Employee1.get_salary())
    print(" ")
```

Enter a number: 2

Employee 1
Enter a name : Ramu
Enter a Employee_salary: 8900
Enter a amount 45
bank_employee_databases
('Ramu', 8900)
RS 45 is credit
total salary is 8945
('Ramu', 8945)
8945

Employee 2
Enter a name : Ramesh
Enter a Employee_salary: 10000
Enter a amount 6788
bank_employee_databases
('Ramesh', 10000)
RS 6788 is credit
total salary is 16788
('Ramesh', 16788)
16788

```
class BankEmployee:

    def __init__(self, full_name, salary):
        self.full_name = full_name
        self.__salary = salary
        print("Bank employee created")

    def display_employee_details(self):
        print(f"Name: {self.full_name}")

    def salary_increment(self, amount):
        if amount > 0:
            self.__salary += amount
```

```
        print(f"Rs {amount} credited")
        print(f"Total salary: {self.__salary}")
    else:
        print("Invalid increment amount")

    def get_salary(self):
        return self.__salary

n = int(input("Enter number of employees: "))
employees = []

for i in range(n):
    print(f"\nEmployee {i+1}")
    name = input("Enter name: ")
    salary = int(input("Enter salary: "))
    amount = int(input("Enter increment amount: "))

    emp = BankEmployee(name, salary)
    employees.append(emp)
    emp.display_employee_details()
    emp.get_salary()
    emp.salary_increment(amount)
    print(f"Final Salary: {emp.get_salary()}")
```

Enter number of employees: 2

Employee 1

Enter name: Bhavesh

Enter salary: 9000

Enter increment amount: 5678

Bank employee created

Name: Bhavesh

Rs 5678 credited

Total salary: 14678

Final Salary: 14678

Employee 2

Enter name: Bholu

Enter salary: 8734

Enter increment amount: 90

Bank employee created

Name: Bholu

Rs 90 credited

Total salary: 8824

Final Salary: 8824

```
class BankEmployee:
```

```
    def __init__(self, full_name, salary):
        self.full_name = full_name
        self.__salary = salary
        print("Bank employee created")
```

```
def display_employee_details(self):
    print(f"Name: {self.full_name}")

def get_salary(self):
    return self.__salary

n = int(input("Enter number of employees: "))
employees = []

for i in range(n):
    print(f"\nEmployee {i+1}")
    name = input("Enter name: ")
    salary = int(input("Enter salary: "))

    emp = BankEmployee(name, salary)
    employees.append(emp)
    emp.display_employee_details()
    emp.get_salary()
    print(f"Final Salary: {emp.get_salary()}")
```

Enter number of employees: 3

Employee 1

Enter name: yuvarj

Enter salary: 7654

Bank employee created

Name: yuvarj

Final Salary: 7654

Employee 2

Enter name: vickey

Enter salary: 7340

Bank employee created

Name: vickey

Final Salary: 7340

Employee 3

Enter name: vinesh

Enter salary: 8900

Bank employee created

Name: vinesh

Final Salary: 8900

```
class bank_account:
```

```
    def __init__(self,account_holder,balance):
```

```
        self.account_holder = account_holder
```

```
        self.__balance = balance
```

```
        print("Bank_account database:")
```

```
    def show_salary_details(self):
```

```
        return f"balance of {self.account_holder} is RS {self.__balance}"
```

```
account_holder = input("Enter name of account_holder : ")
balance = int(input("Enter account_holder balance : "))
acc1 = bank_account(account_holder,balance)
print(acc1.show_salary_details())
```

```
Enter name of account_holder : kunal
Enter account_holder balance : 5609
Bank_account database:
balance of kunal is RS 5609
```

```
class bank_account:

    def __init__(self,account_holder,balance):
        self.account_holder = account_holder
        self.__balance = balance
        print("Bank_account database:")

    def show_salary_details(self):
        return f"balance of {self.account_holder} is RS {self.__balance}"

    def get_balance(self):
        return f"find balance : {self.__balance}"

account_holder = input("Enter name of account_holder : ")
balance = int(input("Enter account_holder balance : "))
acc1 = bank_account(account_holder,balance)
print(acc1.show_salary_details())
print(acc1.get_balance())
```

```
Enter name of account_holder : kailsh
Enter account_holder balance : 6700
Bank_account database:
balance of kailsh is RS 6700
find balance : 6700
```

```
class bank_account:

    def __init__(self,account_holder,balance):
        self.account_holder = account_holder
        self.__balance = balance
        print("Bank_account database:")

    def show_salary_details(self):
        return f"balance of {self.account_holder} is RS {self.__balance}"

    def get_balance(self):
        return f"find balance : {self.__balance}"

    def deposit(self,amount):
        if amount > 0:
```

```

        self.__balance += amount
        print(f"Rs {amount} is credit")
        print(f"final amount : {self.__balance}")
    else:
        print(f"invalid salary is {self.__balance} ")

def withdrawal(self,amount):
    if amount < self.__balance:
        self.__balance -= amount
        print(f"Rs {amount} is debited")
        print(f"final amount : {self.__balance}")
    else:
        print(f"insuffecient salary is {self.__balance}")

account_holder = input("Enter name of account_holder : ")
balance = int(input("Enter account_holder balance : "))
amount = int(input("Enter deposit amount : "))
acc1 = bank_account(account_holder,balance)
print(acc1.show_salary_details())
print(acc1.get_balance())
acc1.deposit(amount)
print(acc1.show_salary_details())
acc1.withdrawal(amount)
print(acc1.show_salary_details())

```

```

Enter name of account_holder : Suraj
Enter account_holder balance : 7654
Enter deposit amount : 6788
Bank_account database:
balance of Suraj is RS 7654
find balance : 7654
Rs 6788 is credit
final amount : 14442
balance of Suraj is RS 14442
Rs 6788 is debited
final amount : 7654
balance of Suraj is RS 7654

```

```

class bank_account:

    def __init__(self,account_holder,balance):
        self.account_holder = account_holder
        self.__balance = balance
        print("Bank_account database:")

    def show_salary_details(self):
        return f"balance of {self.account_holder} is RS {self.__balance}"

    def get_balance(self):
        return f"find balance : {self.__balance}"

```

```

def deposit(self,amount):
    if amount > 0:
        self.__balance += amount
        print(f"Rs {amount} is credit")
        print(f"final amount : {self.__balance}")
    else:
        print(f"invalid salary is {self.__balance} ")

def withdrawal(self,amount):
    if amount < self.__balance:
        self.__balance -= amount
        print(f"Rs {amount} is debited")
        print(f"final amount : {self.__balance}")
    else:
        print(f"insuffecient salary is {self.__balance}")

n = int(input("Enter number of employee : "))

for i in range(n):
    print(f"number of employee : = {i+1}")
    account_holder = input("Enter name of account_holder : ")
    balance = int(input("Enter account_holder balance : "))
    deposit_amount = int(input("Enter deposit amount : "))
    withdrawl_amount = int(input("Enter Witdrwal amount : "))
    acc1 = bank_account(account_holder,balance)
    print(acc1.show_salary_details())
    print(acc1.get_balance())
    acc1.deposit(deposit_amount)
    print(f"amount after deposit : {acc1.show_salary_details()}")
    acc1.withdrawal(withdrawl_amount)
    print(f"amount after withdrawl: {acc1.show_salary_details()}")
    print(" ")

```

```

Enter number of employee : 2
number of employee : = 1
Enter name of account_holder : Raju
Enter account_holder balance : 7655
Enter deposit amount : 890
Enter Witdrwal amount : 567
Bank_account database:
balance of Raju is RS 7655
find balance : 7655
Rs 890 is credit
final amount : 8545
amount after deposit : balance of Raju is RS 8545
Rs 567 is debited
final amount : 7978
amount after withdrawl: balance of Raju is RS 7978

number of employee : = 2
Enter name of account_holder : Ravi
Enter account_holder balance : 8900

```



```

Enter deposit amount : 654
Enter Withdrawal amount : 7899
Bank_account database:
balance of Ravi is RS 8900
find balance : 8900
Rs 654 is credit
final amount : 9554
amount after deposit : balance of Ravi is RS 9554
Rs 7899 is debited
final amount : 1655
amount after withdrawal: balance of Ravi is RS 1655

```

```

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.__marks = marks    # private variable (encapsulation)

    def calculate_average(self, avg):
        self.avg = avg
        total = sum(self.__marks)
        avg = total / len(self.__marks)
        return avg

    def show_details(self):
        return f"Student: {self.name}, Average Marks: {self.calculate_average()}"

```

```

s1 = Student("Rahul", [80, 90, 85])
print(s1.show_details())

```

```

-----
TypeError                                Traceback (most recent call last)
/tmp/ipython-input-3341762242.py in <cell line: 0>()
      1 s1 = Student("Rahul", [80, 90, 85])
----> 2 print(s1.show_details())
      3

/tmp/ipython-input-2471574924.py in show_details(self)
     11
     12     def show_details(self):
----> 13         return f"Student: {self.name}, Average Marks:
{self.calculate_average()}"
     14

TypeError: Student.calculate_average() missing 1 required positional argument:
'avg'

```

Next steps: [Explain error](#)

```
class student:

    def __init__(self,name,marks):
        self.name = name
        self.marks = marks
        print("Student database ")

    def calculated_avg(self):
        return sum( self.marks)/len( self.marks)

    def get_marks(self):
        return self.marks
```

```
n = int(input("Enter number of students: "))

students = []

for i in range(n):
    print(f"\nStudent number : {i+1}")

    name = input("Enter student name: ")

    marks = []
    subjects = int(input("Enter number of subjects: "))

    for j in range(subjects):
        m = int(input(f"Enter marks of subject {j+1}: "))
        marks.append(m)

    s1 = student(name, marks)
    students.append(s1)

    print(s1.calculated_avg())
    print(s1.get_marks())
```

Enter number of students: 2

Student number : 1
Enter student name: Puskar
Enter number of subjects: 3
Enter marks of subject 1: 78
Enter marks of subject 2: 89
Enter marks of subject 3: 90
Student database
85.66666666666667
[78, 89, 90]

Student number : 2
Enter student name: Akshay

```

Enter number of subjects: 3
Enter marks of subject 1: 76
Enter marks of subject 2: 78
Enter marks of subject 3: 34
Student database
62.666666666666664
[76, 78, 34]

```

```

class student:

    def __init__(self,name,marks):
        self.name = name
        self.marks = marks
        print("Student dashbase")

    def calculated_avg(self):
        avg = sum(self.marks)/len(self.marks)
        return avg

    def show_details(self):
        return f"name of student is {self.name} : avg of student is {self.calculated_avg}"

```

```

s1 = student("Raj",[89,90])
s1.show_details()

```

```

Student dashbase
'name of student is Raj : avg of student is 89.5'

```

```

n = int(input("Enter number of student "))

students = []

for i in range(n):
    print(f"number of student is {i+1}")
    name = input("Enter a name ")
    marks = []
    subject_marks = int(input("enter number of subject:"))
    for j in range (subject_marks):
        m = int(input(f"Enter numner of marks each subject is {j+1} : "))
        marks.append(m)

    s2 = student(name,marks)
    students.append(s2)
    print(s2.show_details())

```

```

Enter number of student 1
number of student is 1
Enter a name Deepak
enter number of subject:3
Enter numner of marks each subject is 1 : 78
Enter numner of marks each subject is 2 : 90

```

```
Enter numner of marks each subject is 3 : 56
Student dashbase
name of student is Deepak : avg of student is 74.66666666666667
```

```
class student:

    def __init__(self,name,marks):
        self.name = name
        self.__marks = marks
        print("Student dashbase")

    def calculated_avg(self):
        avg = sum(self.marks)/len(self.marks)
        return avg

    def show_details(self):
        return f"name of student is {self.name} : avg of student is {self.calculated_av

n = int(input("Enter number of student "))

students = []

for i in range(n):
    print(f"number of student is {i+1}")
    name = input("Enter a name ")
    marks = []
    subject_marks = int(input("enter number of subject:"))
    for j in range (subject_marks):
        m = int(input(f"Enter numner of marks each subject is {j+1} : "))
        marks.append(m)

    s2 = student(name,marks)
    students.append(s2)
    print(s2.show_details())
```